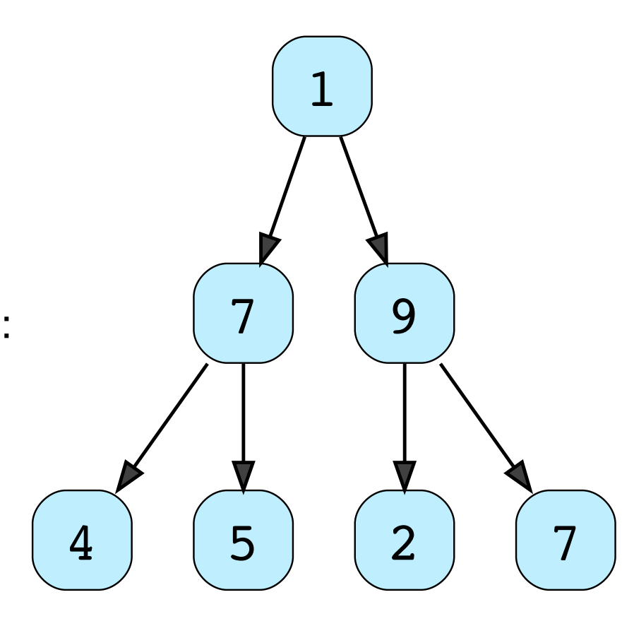
Solution: All Paths for a Sum

Problem Statement

Given a binary tree and a number ‘S’, find all paths from root-to-leaf such that the sum of all the node values of each path equals ‘S’.

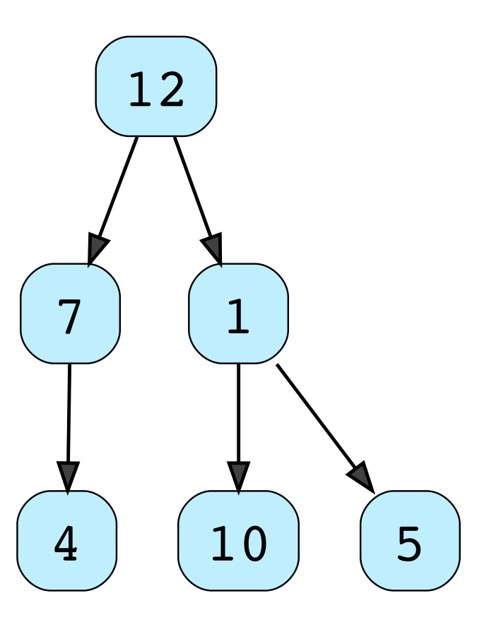
Example 1:

S: 12
Output: [[1, 7, 4], [1, 9, 2]]
Explanation: Here are the two paths with sum '12':
1 -> 7 -> 4 and 1 -> 9 -> 2



Example 2:

S: 23
Output: [[12, 7, 4], [12, 1, 10]]
Explanation: Here are the two paths with sum '23':
12 -> 7 -> 4 and 12 -> 1 -> 10

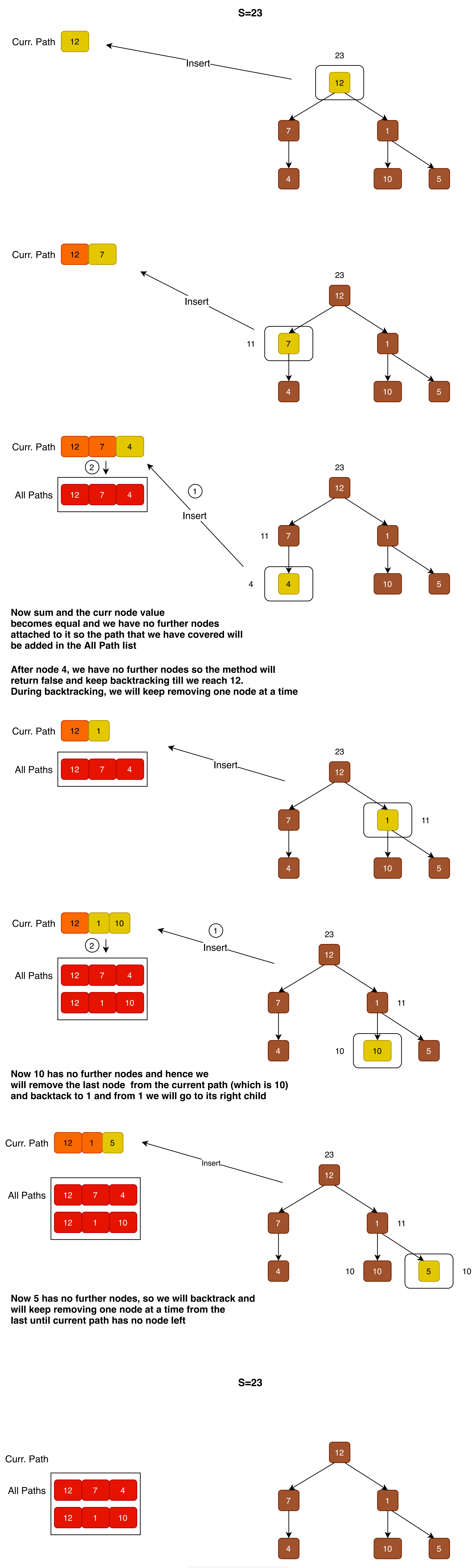


Solution

This problem follows the Binary Tree Path Sum problem. We can follow the same DFS approach. There will be two differences though:

Every time we find a root-to-leaf path, we will store it in a list. We will traverse all paths and will not stop processing after finding the first path.

Here is the visual representation of the algorithm:



Now sum and the curr node value becomes equal and we have no further nodes attached to it so the path that we have covered will be added in the All Path list

After node 4, we have no further nodes so the method will return false and keep backtracking till we reach 12.
During backtracking, we will keep removing one node at a time

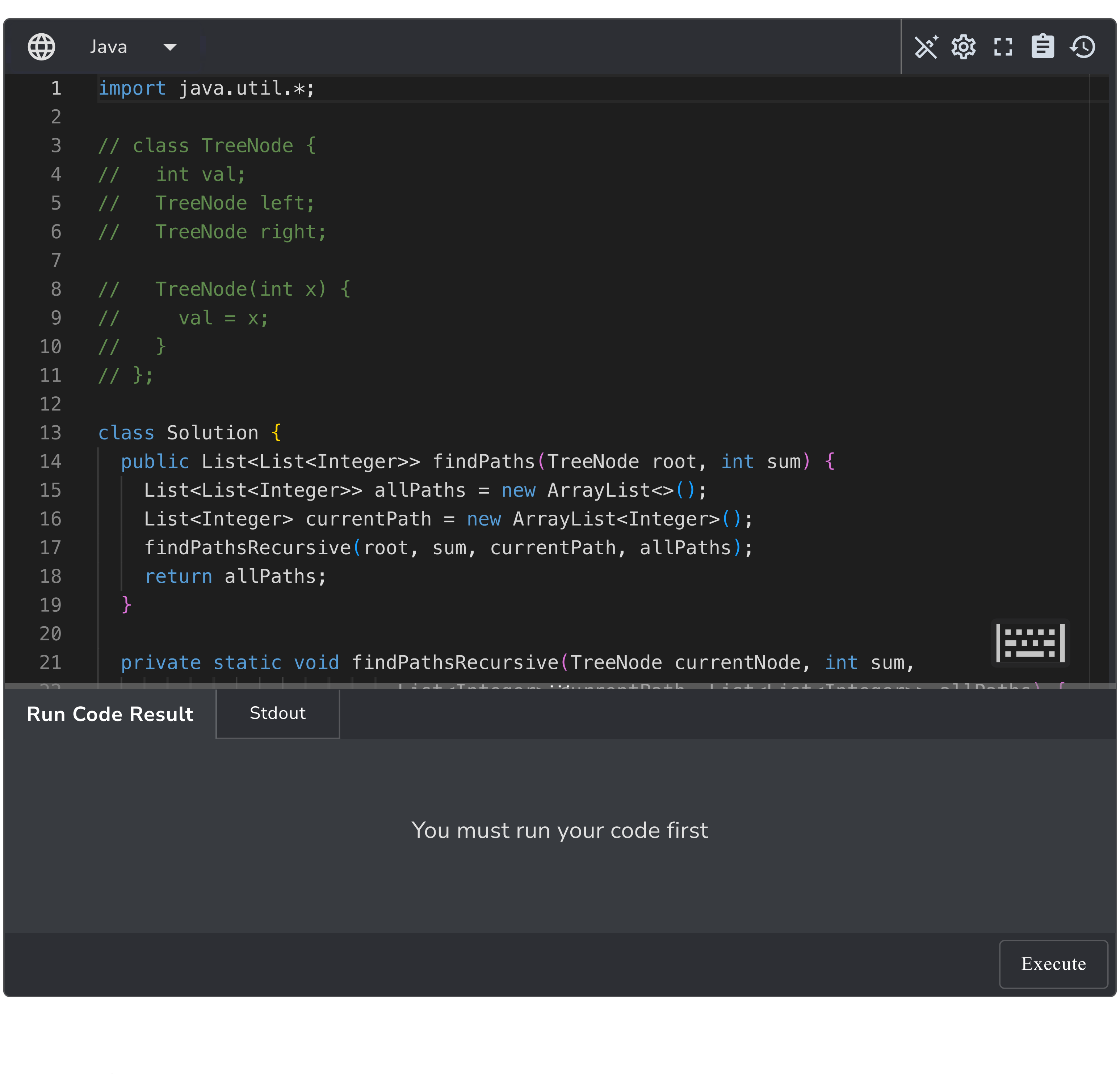
Now 10 has no further nodes and hence we will remove the last node from the current path (which is 10) and backtack to 1 and from 1 we will go to its right child

Now 5 has no further nodes, so we will backtrack and will keep removing one node at a time from the last until current path has no node left

All Paths for a Sum

Code

Here is the code for this algorithm:



```java
import java.util.*;

// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;

//   TreeNode(int x) {
//     val = x;
//   }
// };

class Solution {
  public List<List<Integer>> findPaths(TreeNode root, int sum) {
    List<List<Integer>> allPaths = new ArrayList<>();
    List<Integer> currentPath = new ArrayList<Integer>();
    findPathsRecursive(root, sum, currentPath, allPaths);
    return allPaths;
  }

  private static void findPathsRecursive(TreeNode currentNode, int sum,
```

You must run your code first

Time Complexity

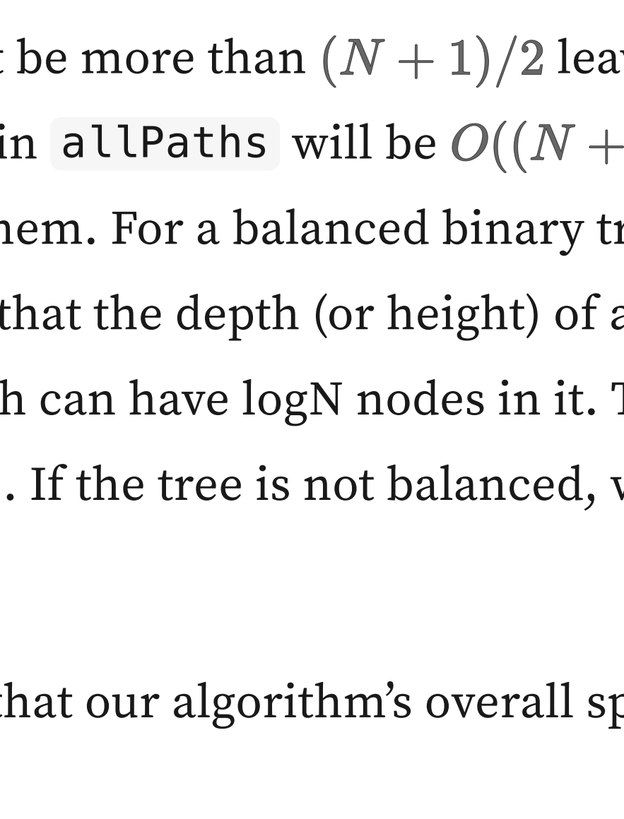
The time complexity of the above algorithm is $O(N^2)$, where ‘N’ is the total number of nodes in the tree. This is due to the fact that we traverse each node once (which will take O(N)), and for every leaf node, we might have to store its path (by making a copy of the current path) which will take O(N).

We can calculate a tighter time complexity of $O(NlogN)$ from the space complexity discussion below.

Space Complexity

If we ignore the space required for the `allPaths` list, the space complexity of the above algorithm will be O(N) in the worst case. This space will be used to store the recursion stack. The worst-case will happen when the given tree is a linked list (i.e., every node has only one child).

How can we estimate the space used for the `allPaths` array? Take the example of the following balanced tree:



Here we have seven nodes (i.e., `N = 7`). Since, for binary trees, there exists only one path to reach any leaf node, we can easily say that total root-to-leaf paths in a binary tree can’t be more than the number of leaves. As we know that there can’t be more than $(N+1)/2$ leaves in a binary tree, therefore the maximum number of elements in `allPaths` will be $O((N+1)/2) = O(N)$. Now, each of these paths can have many nodes in them. For a balanced binary tree (like above), each leaf node will be at maximum depth. As we know that the depth (or height) of a balanced binary tree is O(logN) we can say that, at the most, each path can have logN nodes in it. This means that the total size of the `allPaths` list will be $O(N * logN)$. If the tree is not balanced, we will still have the same worst-case space complexity.

From the above discussion, we can conclude that our algorithm’s overall space complexity is $O(N * logN)$.

Also, from the above discussion, since for each leaf node, in the worst case, we have to copy log(N) nodes to store its path; therefore, the time complexity of our algorithm will also be $O(N * logN)$.

Similar Problems

Problem 1: Given a binary tree, return all root-to-leaf paths.

Solution: We can follow a similar approach. We just need to remove the “check for the path sum.”

Problem 2: Given a binary tree, find the root-to-leaf path with the maximum sum.

Solution: We need to find the path with the maximum sum. As we traverse all paths, we can keep track of the path with the maximum sum.

← Previous
All Paths For A Sum (Medium)
Next →
Sum Of Path Numbers (Medium)
☐ Mark as Completed